Learning Objectives

The primary objective remains to practice the development workflow

edit (and save) → compile → run

assuming compilation succeeds.

Closely related to this is becoming comfortable with the Unix command line. Throughout the course, the shell is used to organize files, invoke the compiler, execute programs, and automate repetitive tasks.

The secondary objective is to develop an intuition for how programs are executed. In particular, you will:

- explore the lifetime of local and global variables,
- understand parameter passing using the call-by-value principle,
- develop a low-level understanding of control flow by reducing high-level control structures such as `if`, `while`, `do...while`, and `for` to simple jumps,
- understand how simple instructions and jumps can describe the step-by-step execution of a program,
- develop a first conceptual model of how a stack can be used to manage local variables during nested and recursive function calls.

In the next worksheets, we will begin building a small processor from the bottom up. We will first construct a Mini-ALU that can perform simple operations on data. But performing individual operations is not enough to execute a program: a processor must also determine which instruction is executed next.

If instructions are stored sequentially in memory, execution can normally continue with the next instruction. Some instructions, however, can change this order by continuing execution at a different instruction, either unconditionally or depending on a condition. These are called unconditional and conditional jumps.

You are not expected to understand how such instructions are implemented yet. In this worksheet, we approach the same idea from the top down by taking familiar programming-language control structures apart. Later, when we encounter jumps again from the hardware side, it may be worthwhile to look back at this worksheet. You may then discover that some of the details introduced here have a surprisingly simple explanation.

Step 1

Throughout the rest of the course, you will create a growing collection of source files, worksheets, and experiments. It is therefore worth spending a few minutes organizing your files from the beginning.

Tasks

Use the shell to create a directory structure for the course. For example, create the following directories inside your home directory:

```
hpc0/
├── session01/
├── session02/
└── session03/
```

Move the files from the previous sessions into their corresponding directories.
You are free to organize your files differently if you prefer. The important point is to develop the habit of keeping projects well organized.

Step 2 — Lifetime of Variables

Variables have not only a **scope** (where they can be accessed) but also a **lifetime** (when they exist).

A **global variable** has **scope** from its declaration until the end of the program. Its **lifetime**, however, begins when the program starts and ends when the program terminates.

A **local variable** has **scope** only within the block in which it is declared. Its **lifetime**, however, begins when the corresponding function is entered and ends when the function returns.

Remark: Notice that the beginning of a variable's scope and the beginning of its lifetime are not necessarily the same.

© Michael C. Lehn | CC BY-SA 4.0

A global variable already exists when the program starts, but it cannot be accessed before its declaration. In other words, its lifetime begins before its scope.

For local variables declared directly in a function body, the situation is different. Their scope and lifetime begin at approximately the same time: the scope begins at the declaration, while the lifetime begins when the function is entered—not when execution reaches the declaration.

```
1 @ <stdio.hdr>
2
3 global a: int = 42;
4
5 fn foo()
6 {
7     local b: int;
8     printf("foo entered: b = %d\n", b);
9     b = a;
10    printf("foo leaving: b = %d\n\n", b);
11 }
12
13 fn main()
14 {
15    // printf("calling foo:\n");
16    foo();
17
18    // printf("calling foo:\n");
19    foo();
20
21 }
```

Tasks

1. Compile and run the program `ex3.abc` shown on the right. On my system, I obtain the following output:

   ```
   MCL:tmp lehn$ ./a.out
   foo entered: b = 1
   foo leaving: b = 42

   foo entered: b = 42
   foo leaving: b = 42
   ```

   Your output may differ because, as we will see, the program exhibits **undefined behavior**.
2. If we carefully trace the control flow of the program and compare it with the output, the result initially appears to contradict the statement above about the lifetime of local variables.
   Go through the program line by line and determine exactly when each line of output is produced.
   Pay particular attention to the following:
   (A) In line 8, the value of the variable `b` is printed before `b` has been initialized. The value printed during the first call to `foo`—`1` on my system—is therefore undefined.
   (B) In line 9, the value of `b` is overwritten with the value of `a`, which is `42`.
   (C) During the second call to `foo`, line 8 once again prints the value of `b` before the variable has been initialized. On my system, the value `42` is printed. This is the value that `b` had at the end of the previous function call. It therefore appears as though the local variable `b` has "survived" the previous call to `foo`.
3. Uncomment the `printf` statement in lines 15 and 18. Compile and run the program again. Two additional lines are now printed. More importantly, however, the output produced by `foo` should also have changed. For example, I obtain:

   ```
   MCL:tmp lehn$ ./a.out
   calling foo:
   foo entered: b = 0
   foo leaving: b = 42

   calling foo:
   foo entered: b = 0
   foo leaving: b = 42
   ```

   We can now observe that `b` did not survive the previous function call after all. Instead, a new local variable `b` was created during the second call to `foo`. It merely appears that this newly created variable was initialized to zero.
4. Compile the program again, this time with optimization enabled. For example[1]:

   ```
   MCL:tmp lehn$ abc -O1 ex3.abc
   ```

   On my system, I obtain the following output:

   ```
   calling foo:
   foo entered: b = 73896
   foo leaving: b = 42

   calling foo:
   foo entered: b = 73896
   foo leaving: b = 42
   ```

[1] Note that a capital O does not display the dot in it's middle, present at zeros - now you can distinguish them easily

© Michael C. Lehn | CC BY-SA 4.0

It now appears as though `b` is initialized to `73896` during every call. However, if the `printf` statements in lines 15 and 18 are commented out again, I obtain:

```
MCL:tmp lehn$ abc -O1 ex3.abc
MCL:tmp lehn$ ./a.out
foo entered: b = -1148205632
foo leaving: b = 42

foo entered: b = 73896
foo leaving: b = 42
```

Conclusion

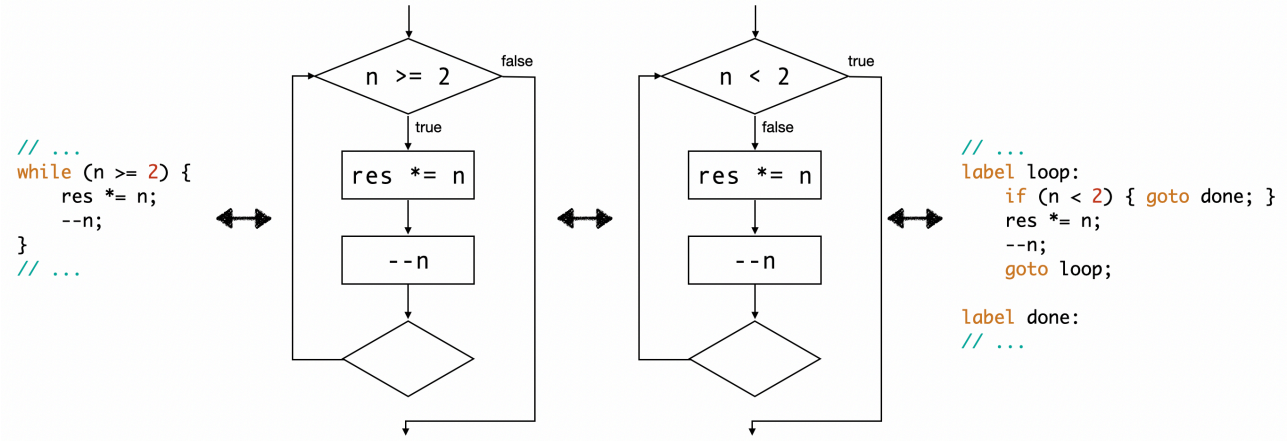
a. Local variables do not survive a function call.
b. Local variables are created anew for every function call and have an undefined value unless they are explicitly initialized.
c. If an undefined value is printed, the output may depend on the date, the time of day, your current mood, and the weather.

Step 3 — Control Flow and `goto`

In the lecture, we saw how **flowcharts** can be used to describe the control flow within a function. The following conventions were used:

- **Conditional branches** are represented by a diamond containing the condition.
- **Unconditional branches** are represented by an empty diamond.
- All other statements are represented by rectangles containing the corresponding statement.
- The diamonds and rectangles are arranged vertically, reflecting the fact that the corresponding machine instructions are stored sequentially in memory. Arrows indicate the order in which the instructions are executed. Execution always starts at the top.

To get a first idea of how high-level control structures can eventually be translated into simple machine instructions and jumps, we also showed how a `while` loop can be implemented using `goto` statements:



Comparing the flowchart with the version using `goto` reveals two simple rules:

- An empty diamond corresponds to an unconditional `goto`.
- A diamond containing a condition corresponds to a statement of the form

  ```
  if (…) { goto … }
  ```

  This construct is often referred to informally as a *conditional goto* or `if`-`goto`.

Important: We are not doing this because `goto` is a better way to write programs. Quite the opposite: high-level control structures such as `if`, `while`, and `for` usually make programs much easier to understand. Here, goto is a tool that helps us expose the underlying control flow.

© Michael C. Lehn | CC BY-SA 4.0

Tasks

You will derive the rules for expressing control structures using goto statements by examining several examples.

1. `if` / `else` — **discover the rules**

   Consider the program `if.abc` below.

```
@ <stdio.hdr>

fn main()
{
    local ch: int = getchar();

    if (ch == 'A') {
        printf("I got an A!\n");
    } else {
        printf("Not an A :(\n");
    }
}
```

   First, run the program with different inputs and make sure that you understand its behavior.

   Then draw a flow chart describing its control flow. Rewrite the flow chart using only the two forms introduced above:

```
goto label;
```

   and

```
if (condition) { goto label; }
```

   Finally, implement this version as if_goto.abc. Verify that if.abc and if_goto.abc behave identically for different inputs.

2. `do`-`while` — **recognize the loop**

   The following program `do_while.abc` repeatedly reads and prints a character until it reads a dot (`.`).

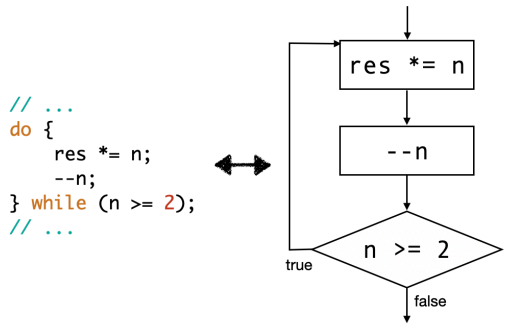
```
@ <stdio.hdr>

fn main()
{
    local ch: int;

    do {
        ch = getchar();
        printf("got '%c' (ASCII %d)\n", ch, ch);
    } while (ch != '.');
}
```

   Run the program and confirm that the final character `.` is printed as well.

   The flow chart below describes a corresponding `do`-`while` loop. Use it to rewrite the program using only `goto` and `if`-`goto`. Save your implementation as `do_while_goto.abc`.



   Compare the resulting control flow with the `if`/`else` example. Which `goto` causes an instruction to be executed again?

© Michael C. Lehn | CC BY-SA 4.0

3. `for` **— take the abstraction apart**

   Finally, consider the program `for.abc` below.

```
@ <stdio.hdr>

fn main()
{
    local n: int = 5;
    local res: int = 1;

    for (local i: int = 1; i <= n; ++i) {
        res *= i;
    }
    printf("res = %d\n", res);
}
```

   Before drawing a flow chart, identify the four parts of the for loop:

   (1) initialization, (2) condition, (3) loop body, and (4) increment.

   Determine the order in which these parts are executed. Then draw the corresponding flow chart and rewrite the program using only goto and if-goto. Save your implementation as for_goto.abc and verify that both programs produce the same result.

A short remark

So far, we have taken a first look behind high-level control structures. In the remaining steps, we will do something similar for function calls. These steps serve two purposes. On the one hand, they introduce additional features of the ABC programming language and provide further programming practice. On the other hand, each experiment reveals a small piece of the implementation of function calls. By carefully observing the behavior of the programs, we will gradually discover how concepts such as parameter passing and local variables can be implemented. You do not need to understand this broader didactic plan yet. Just keep experimenting—the neurons responsible for understanding function calls are already being prepared. 😉

Step 4 — Call by Value

When an argument is passed to a function, the function receives a copy of that argument. The original argument therefore remains unchanged after the function call, even if the called function overwrites its local copy.

```
 1 @ <stdio.hdr>
 2
 3 fn foo(n: int)
 4 {
 5     printf("foo entered: n = %d\n", n);
 6     n = 42;
 7     printf("foo leaving: n = %d\n", n);
 8 }
 9
10 fn main()
11 {
12     local n: int = 123;
13
14     printf("main: n = %d\n", n);
15     printf("calling foo:\n");
16     foo(n);
17     printf("returned from foo\n");
18     printf("main: n = %d\n", n);
19 }
```

Tasks

1. Compile and run the program. Go through it line by line and determine which line produces each part of the output. Use your observations to confirm that the function receives only a copy of the argument rather than the original variable.
2. To emphasize that the caller and the callee may use different names for the argument, rename the local variable `n` in main to `x`. Compile and run the program again. Its behavior should remain unchanged.

© Michael C. Lehn | CC BY-SA 4.0

Step 5 — Using a Stack for Function Calls

A stack is a data structure that supports two basic operations:
- A push operation places a new value on top of the stack.
- A pop operation removes the topmost value from the stack.

A stack can be used to manage the data associated with local variables during function calls. As we will see later, it also stores the information needed to return from a function call to the correct location in the program.
The compiler automatically generates the code that manipulates the stack, so these operations are already part of the executable program. In the Bottom-Up sessions of this course, we will write this code ourselves in assembly language and thereby gain a much deeper understanding of what the compiler is doing on our behalf.
For now, our goal is simply to understand what this generated code does rather than how it is implemented. To do so, we will analyze the following example program:

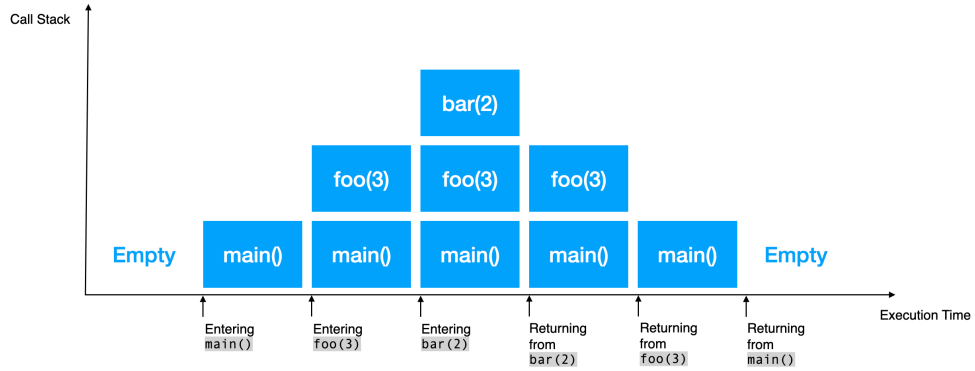
```
1 @ <stdio.hdr>
2
3 fn bar(n: int)
4 {
5     printf("bar entered: n = %d\n", n);
6 }
7
8 fn foo(n: int)
9 {
10    printf("foo entered: n = %d\n", n);
11    bar(n - 1);
12    printf("foo leaving: n = %d\n", n);
13 }
14
15 fn main()
16 {
17    foo(3);
18 }
```

Tasks

1. Compile and run the program. Go through it line by line and determine which line produces each part of the output.

   In particular, make sure you understand the following:
   - The parameter `n` in `foo` still exists after the call to `bar` and retains its value.
   - The function `bar` also has a parameter called `n`, but this is a different variable from the one in `foo`.

2. The following diagram illustrates how a stack can be used to manage function calls:



   For each function call, a new box is created containing the relevant data. Such a box is called a stack frame. This data includes, in particular, the local variables—including the function parameters—and, as mentioned above, the return address. These boxes are placed on top of one another:
   (A) When the program starts, the stack is empty.
   (B) When `main` is entered, a new box is created and pushed onto the stack.
   (C) When `main` calls `foo` with the argument `3`, another box is created and pushed onto the stack.

© Michael C. Lehn | CC BY-SA 4.0

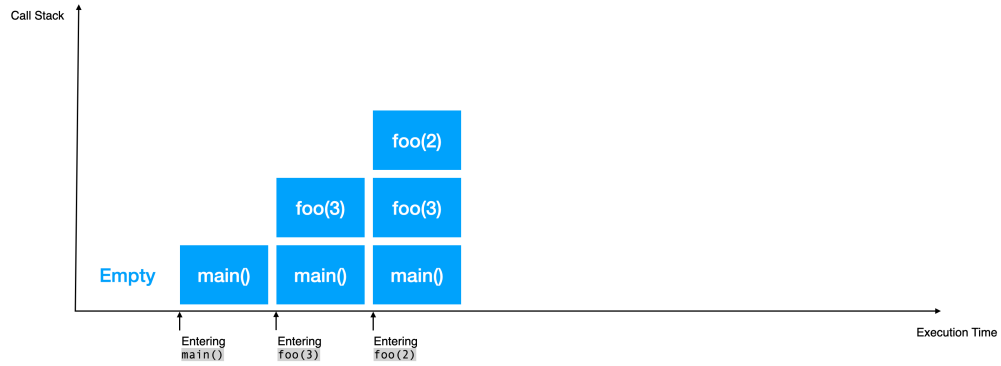
(D) When `foo` calls `bar` with the argument `2`, yet another box is created and pushed onto the stack.
(E) When `bar` returns, the topmost box is removed. When execution continues in `foo`, the topmost box once again contains the state of `foo` from before the call to `bar`.
(F) And so on.

Task

Stack diagrams can also be used to illustrate recursive function calls. Create such a diagram for the following program:

```
1 @ <stdio.hdr>
2
3 fn foo(n: int): int
4 {
5     local res: int = n;
6
7     if (n > 1) {
8         res += foo(n - 1);
9     }
10    return res;
11 }
12
13 fn main()
14 {
15    printf("foo(%d) = %d\n", 3, foo(3));
16 }
```

Include the current values of all local variables in each box. The incomplete diagram shown below illustrates the first few function calls:



Extend the program by adding `printf` statements that trace when functions are entered and left. Compare the generated output with your stack diagram and verify that the sequence of function calls matches your drawing.

Make the following changes:

- At the beginning of `main`, print `Entering main()`
- Immediately before returning from `main`, print `Returning from main()`
- At the beginning of `foo`, print Entering `foo(%d)` using the current value of the parameter `n`.
- Before the `return` statement in `foo`, print `Returning value %d from foo(%d)` using the current values of `res` and `n`.

After these modifications, running the program should produce a trace similar to the following:

```
Entering main()
Entering foo(3)
Entering foo(2)
Entering foo(1)
Returning value 1 from foo(1)
Returning value 3 from foo(2)
Returning value 6 from foo(3)
Returning from main()
```

© Michael C. Lehn | CC BY-SA 4.0